Rigid Balls, Round walls

Rigid Ball Collisions in a Circular Boundary

Gravity integration and collision detection for rigid balls bouncing inside a circular boundary.

Simulations
SEDS Celestia
Crew Member Induction

Due: EOD, 7th June 2026 **Prerequisites:** Basic Python, elementary mechanics

This assignment is arranged in tasks of increasing difficulty. You are expected to attempt all of them and submit the Google Form with your GitHub repository linked. Partial submissions are read: genuine effort and clear thinking count for more than a finished notebook that you cannot explain. **Along with the code, write a short assignment brief in your repository's `README.md`: answer the two questions marked in the text, and say what you make of your own results. Half a page is plenty; this should not take you long.**

Introduction. A single ball falling under gravity is a first-week exercise. A room full of them, each bouncing off the walls and off each other, is where it gets interesting: you have to decide when two bodies are touching, and what happens the instant they do.

You will build a two-dimensional rigid-ball simulator in Python and draw it with Pygame: circular bodies of equal radius, falling under gravity inside a circular boundary. Get one ball bouncing correctly off the wall first, then two balls bouncing off each other without passing through one another. Many balls at once is left as a bonus.

If you have not used Pygame before, there will be instructions in the `README.md` for setup. You need very little of it here: a window, a clock to fix the frame rate, and `pygame.draw.circle` to draw each ball and the boundary.

A Note on Notation

Two conventions are used throughout.

A subscript on a bold symbol advances the whole vector. A bold symbol is a stack of numbers, and the subscript n counts timesteps of length Δt. Bumping it by one advances every component at once:

$$\mathbf{S}_n = \begin{bmatrix} x_n \\ y_n \\ v_{x,n} \\ v_{y,n} \end{bmatrix} \quad \longmapsto \quad \mathbf{S}_{n+1} = \begin{bmatrix} x_{n+1} \\ y_{n+1} \\ v_{x,n+1} \\ v_{y,n+1} \end{bmatrix}, \qquad t_{n+1} = t_n + \Delta t.$$

So $\mathbf{S}_{n+1}$ is the entire state one timestep later, not a single entry of it. A second subscript, where it appears, labels the ball: $\mathbf{v}_{i,n}$ is the velocity of ball i at step n.

The left arrow is assignment, not equality.

$$a \leftarrow f(a) \qquad \text{means} \qquad a_{\text{new}} = f(a_{\text{old}}), \qquad \text{in Python: } \texttt{a = f(a)}.$$

It is written this way because a collision overwrites the velocity in place at a single instant, with no timestep in between, so $a = f(a)$ would be false as an equation. For example,

$$\mathbf{v}_i \leftarrow \mathbf{v}_i - (1 + e_w)(\mathbf{v}_i \cdot \hat{n})\,\hat{n}$$

reads: compute the right-hand side from the current $\mathbf{v}_i$, then store the result back into $\mathbf{v}_i$.

I. Setting Up the System

Each ball i carries a state (position, velocity). All balls share the same fixed radius ρ and the same fixed mass m:

$$\mathbf{S}_i = \begin{bmatrix} x_i \\ y_i \\ v_{x,i} \\ v_{y,i} \end{bmatrix}, \qquad \text{radius } \rho, \quad \text{mass } m.$$

Between collisions, every ball falls freely under standard gravity:

$$\frac{d\mathbf{r}_i}{dt} = \mathbf{v}_i \tag{1}$$

$$\frac{d\mathbf{v}_i}{dt} = \mathbf{g}, \qquad \mathbf{g} = (0, -g), \quad g = 9.8 \text{ m s}^{-2} \tag{2}$$

The arena is a circle of radius R centred at the origin. A ball's centre is constrained to $|\mathbf{r}_i| \le R - \rho$. On contact with the wall, reflect the velocity about the outward radial normal $\hat{n} = \mathbf{r}_i / |\mathbf{r}_i|$, with a wall restitution coefficient e_w:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i - (1 + e_w)(\mathbf{v}_i \cdot \hat{n})\,\hat{n}$$

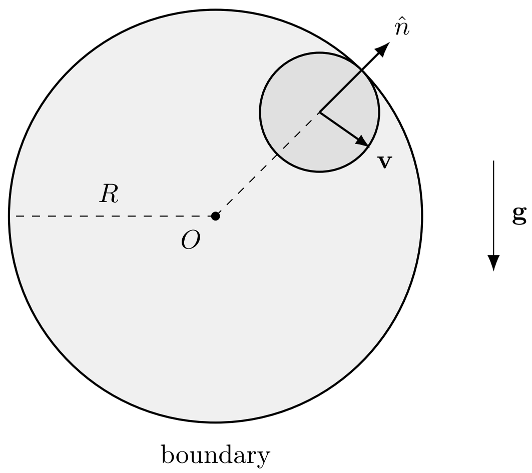

Figure 1. A single ball inside the arena. The contact normal $\hat{n}$ points straight out from the centre O through the ball, so it is found by normalising the ball's own position vector. The velocity $\mathbf{v}$ is reflected about this direction on contact.

Part A. Building the Model. Implement free-fall integration using Euler's method: update the velocity first, then use that updated velocity to move the position, every step.

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{g}\,\Delta t, \qquad \mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1}\,\Delta t$$

Then add the wall-collision response above, applied whenever a ball crosses the boundary.

Part B. Getting It Right. Run a single ball in an empty arena and watch it bounce. Check that changing e_w behaves as you would expect: at $e_w = 1$ every bounce should return to the same height, and at $e_w < 1$ each bounce should be visibly lower than the last.

> ***Question 1.*** *A fast enough ball can cross the boundary within a single timestep, so a check that only asks "is the ball outside right now?" never sees it there at all. Which of Δt, g and R make this more likely, and what would you test each step instead?*

> ***Question 2.*** *Set $e_w = 1$, so that no energy is lost at a bounce, and let the ball run for a few thousand steps. Does the peak height stay put, creep upward, or decay? Gravity and the bounce rule are the only things acting, so if it changes at all, where is that energy coming from?*

II. Two Balls Colliding

Now for the case that actually involves two bodies: a single collision between two balls. The contact normal is found the same way as at the wall, just between two moving bodies instead of a ball and a fixed boundary.

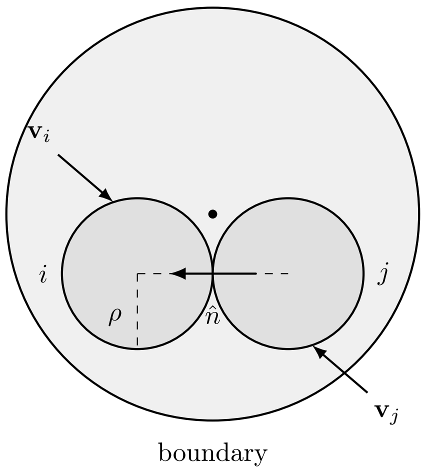

Figure 2. Two balls in contact inside the arena. $\hat{n}$ points along the line joining their centres; both share the same radius ρ and mass m. Only the velocity components along $\hat{n}$ change.

Two balls i, j are in contact when

$$|\mathbf{r}_i - \mathbf{r}_j| < 2\rho$$

Resolve the collision along the line of centres, $\hat{n} = (\mathbf{r}_i - \mathbf{r}_j)/|\mathbf{r}_i - \mathbf{r}_j|$. With restitution e and normal relative velocity $v_n = (\mathbf{v}_i - \mathbf{v}_j) \cdot \hat{n}$, resolve only when $v_n < 0$ (the balls are approaching). Because both balls have the same mass, the mass cancels out entirely and each ball simply takes an equal and opposite share of the correction:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i - \tfrac{1}{2}(1+e)\,v_n\,\hat{n}, \qquad \mathbf{v}_j \leftarrow \mathbf{v}_j + \tfrac{1}{2}(1+e)\,v_n\,\hat{n}$$

For a head-on elastic collision ($e = 1$) this gives the familiar result: the two balls exchange their velocity components along $\hat{n}$, and leave the perpendicular components untouched.

Simulate exactly two balls on a collision course and confirm that they bounce apart rather than passing through each other.

III. Bonus

[bonus] Everything below is optional. Attempt it only once the two required sections above actually work.

Many Balls. Extend the simulation from two balls to N of them. Check every pair for contact using the same rule as before, and resolve each contact the same way. You will also need a positional correction: push overlapping balls apart along $\hat{n}$, moving each by half the overlap, so they do not sink into one another over successive steps. Then push N as high as you can get it while the simulation still runs at a reasonable speed, and note in your brief the number you reached and what slowed you down.

Submission

A single public GitHub repository, linked in the Google Form, containing:

- Source code.
- A `README.md` holding your assignment brief. It should contain answers to the questions (**mandatory**) and can contain any personal remarks, comments, struggles, where it failed, etc. (**extra, but it'll provide you more brownie points if you do so!**)
- Any extra artefact you want: notebook, animation, writeup.

> *Incomplete work submitted with honest notes on where it broke scores above complete work you cannot explain.*